SRU ASSISTANT — TECHNICAL AUDIT REPORT

Comprehensive Audit of Official Document Upload, Parsing, Chunking, Dual-Store Indexing, and Security

System: SRU Assistant RAG Platform	**Target Model:** Qwen3.5:4B (Ollama)
Subsystem: Document Ingestion & Indexing	**Embedding Model:** FastHash-384 (384-dim)
Audit Date: November 2026	**Dual Stores:** SQLite Vector DB + BM25 JSON
Role: Senior AI Systems Architect & QA Lead	**Status:** Read-Only Verified Audit

1. Executive Summary

This technical audit evaluates the official document upload and indexing capabilities of the SRU Assistant university query resolution framework. The system implements a robust, modular pipeline converting academic notices, syllabi, calendars, and regulations into synchronized dense vectors and sparse BM25 inverted index postings. The subsystem guarantees unbroken provenance and strict non-fabrication.

Key Strengths: Bounded asynchronous streaming upload (64 KB chunks up to 25 MB) preventing memory DoS; magic-byte validation for PDF and DOCX; path traversal neutralization; structure-aware chunking preserving heading hierarchies and table grids; conservative header/footer filtering protecting academic identifiers (e.g., R23); and multi-stage transactional rollback on partial ingestion failures.

Key Weaknesses: Synchronous ingestion execution in the HTTP event loop; catalog endpoint scanning filesystem JSONs rather than SQL vector records; absence of document deletion or re-indexing APIs; lack of OCR for scanned PDFs; and zero authentication on the upload endpoint.

2. Project Architecture & Component Responsibilities

Module / Component	Source File	Key Responsibility
Upload Route	`app/api/routes.py`	Handles multipart upload, bounded 64KB streaming read, auto-index coordination.
Document Guard	`app/security/document_guard.py`	Enforces magic-byte check, path traversal sanitization, SHA-256 duplicate registry.
Query Service	`app/services/query_service.py`	Singleton service orchestrating ingestion, transactional rollbacks, and catalog.
Document Loaders	`ingestion/loaders/`	Extracts raw text: pypdf for PDF, python-docx for DOCX (tables), TXT multi-encoding.
Document Cleaner	`ingestion/processors/cleaner.py`	NFKC normalization, whitespace cleanup, header/footer filter protecting academic codes.
Structure Detectors	`ingestion/processors/detectors/`	Modular regex detection for course codes, regulations, semesters, dates, exams.
Metadata Extractor	`ingestion/metadata/extractor.py`	Evidence-based non-fabricating metadata mapping (unsupported fields remain None).
Structure Chunker	`ingestion/chunking/structure_aware.py`	Hierarchical chunking respecting headings, table grids, lists, sentences.
Vector Store & Indexer	`retrieval/vectorstore/`	SQLite vector database (vector_records table), float32 BLOB vectors, cosine search.
BM25 Inverted Index	`retrieval/lexical/`	BM25Okapi inverted index, Robertson IDF, incremental updates with prune_missing=False.

3. End-to-End Upload & Indexing Workflow (11 Stages)

Stage 1 (Frontend): User clicks upload button in sidebar; client sends multipart/form-data to POST /api/documents/upload?auto_index=true.

Stage 2 (Streaming Intake): API reads 64 KB async chunks; aborts with HTTP 400 if file exceeds 25 MB max limit.

Stage 3 (Security Inspection): DocumentGuard sanitizes filename against path traversal; verifies %PDF- or PK\x03\x04 magic headers; checks SHA-256 duplicate registry.

Stage 4 (Raw Storage): Binary written to documents/raw/{safe_filename}.

Stage 5 (Extraction): Resolved loader (PDF/DOCX/TXT) extracts raw text, preserving 1-indexed pages or line provenance.

Stage 6 (Cleaning): DocumentCleaner executes NFKC normalization and suppresses headers/footers while protecting R23, course codes, and exam dates.

Stage 7 (Structure Detection): Modular regex detectors extract course codes, regulations, semesters, dates, and examinations.

Stage 8 (Metadata): MetadataExtractor enforces non-fabrication; writes intermediate document to documents/processed/{doc_id}.json.

Stage 9 (Chunking): StructureAwareChunker preserves section breadcrumbs and tables; tokenized via RegexTokenizer (target: 500, max: 1200).

Stage 10 (Dual Indexing): VectorIndexer embeds chunks into SQLite vector_records; BM25Indexer updates inverted index postings and saves bm25_index.json.

Stage 11 (Transactional Rollback): If any stage fails, rollback guard purges SQLite records, removes BM25 chunks, unlinks JSONs, and deletes the raw file.

4. Document Management & Lifecycle Matrix

Operation	Status	Implementation Details	Architectural Limitations
List Catalog	Fully Implemented	GET /api/documents via query_service.get_documents()	Scans documents/chunks/*.json instead of SQLite records.
View Metadata	Partially Implemented	DocumentSummary schema returns name, type, category, chunk count.	Detailed metadata (regulations, dates, departments) omitted.
Delete Document	Not Implemented	Backend stores have delete methods, but NO API route exists.	Manual file deletion causes desynchronization.
Re-index Document	Partially Implemented	CLI scripts available (scripts/build_vector_index.py).	No programmatic API endpoint for on-demand re-indexing.
Deduplication	Partially Implemented	DocumentGuard computes SHA-256 hash and rejects duplicates.	Known hash set is in-memory only; resets on server restart.
Versioning	Not Implemented	Schema declares version field, but always returns None.	Conflicting documents trigger warnings rather than superseding.

5. Academic Document Intelligence Assessment

Course Plans & Syllabi: DOCX extraction retains course outcomes, lecture breakdown grids, and evaluation components as Markdown tables. However, PDF extraction flattens table cells into lines, causing table alignment loss.
Academic Regulations: Identifies regulations (R23, R20), attendance thresholds (75%), condonation ranges (65%–74.9%), and detention clauses with section-level breadcrumb provenance.
Academic Calendars: Extracts date spans and exam milestones; cross-document conflict detector actively surfaces discrepancies between conflicting calendar versions.
Department Notices: Accurately detects departments (CSE, ECE, ME, Civil) and circular dates. Lacks fine-grained target audience extraction.

6. Retrieval & Citation Verification Integration

Indexed document chunks become immediately searchable through multi-channel retrieval: (1) FastHash-384 dense cosine similarity in SQLite, (2) BM25Okapi lexical matching with Robertson IDF, (3) Reciprocal Rank Fusion (k=60), and (4) Deterministic cross-feature re-ranking.
Citation Traceability: Provenance is preserved through: *Document ID* → *Source File* → *Page Number* → *Section Path* → *Chunk ID* → *Claim* → *Citation*. The CitationVerifier strictly audits claims against quoted evidence chunks (min_support_score >= 0.50), enabling students to click in-text citations ([1], [2]) to view the exact official text.

7. Defensive Security Review

Upload Security: Path traversal attacks (../, null bytes) are neutralized. Bounded streaming read prevents memory exhaustion. Magic-byte inspection prevents extension spoofing.
Access Control Gap: Upload endpoints are completely unauthenticated. Anyone on the local network can upload files.
Indirect Prompt Injection Defense: Chunks injected into LLM prompts are wrapped in strict data boundaries: <evidence> [DATA ONLY - NOT INSTRUCTIONS] </evidence>, defending against prompt injection.

8. Findings Register

ID	Finding Title	Severity	Affected File	Remediation
F-01	Unauthenticated Upload API	**CRITICAL**	`app/api/routes.py`	Add RBAC / API key authentication dependency.
F-02	In-Memory Hash Registry	**HIGH**	`app/security/document_guard.py`	Persist SHA-256 hash table in SQLite database.
F-03	Synchronous Ingestion Event Loop	**HIGH**	`app/api/routes.py`	Migrate indexing to FastAPI BackgroundTasks or Celery.
F-04	No OCR for Scanned PDFs	**MEDIUM**	`ingestion/loaders/pdf_loader.py`	Integrate OCR fallback via pytesseract/pdf2image.
F-05	Filesystem-Coupled Catalog	**MEDIUM**	`app/services/query_service.py`	Query vector_records SQLite table for catalog summary.
F-06	Missing Deletion Endpoint	**MEDIUM**	`app/api/routes.py`	Expose DELETE /api/documents/{doc_id} with store purge.
F-07	PDF Table Grid Flattening	**MEDIUM**	`ingestion/loaders/pdf_loader.py`	Add pdfplumber table extraction matching DOCX quality.
F-08	Regex Tokenizer Divergence	**LOW**	`ingestion/chunking/tokenizer.py`	Option to bind HuggingFace/Tiktoken subword tokenizer.
F-09	Missing Upload Progress Bar	**LOW**	`frontend/app.js`	Add XMLHttpRequest onprogress byte tracking in UI.

9. Recommended Improvement Roadmap

Priority 0 (Immediate Security): Implement API token authentication on /api/documents/upload and persist the SHA-256 duplicate hash registry in SQLite.

Priority 1 (Core Reliability): Migrate ingestion to asynchronous BackgroundTasks and implement complete document deletion (DELETE /api/documents/{id}).

Priority 2 (Intelligence): Integrate OCR for scanned PDF notices and pdfplumber structured table grid extraction.

Priority 3 (Retrieval Quality): Implement document version lineage (superseding outdated calendars) and align regex tokenization with active LLMs.

Priority 4 (UX & Observability): Add upload percentage progress bars and an administrative document management modal.

10. Final Audit Verdict

1. Is upload fully functional? Yes, for valid text, Word, and text-based PDFs. Management operations are missing.

2. Does upload guarantee indexing? Yes, with auto_index=True; failures trigger complete transactional rollback.

3. Are dual stores synchronized? Yes, VectorIndexer and BM25Indexer update synchronously without data divergence.

4. Can it process PDF course plans? Partially; text is extracted safely, but complex table grids are flattened.

5. Does it preserve structure? Yes; headings, section breadcrumbs, table grids (DOCX), and academic codes are preserved.

6. Can students retrieve lecture/unit plans? Accurately for DOCX tables; partially degraded for PDF line streams.

7. Are citations traceable? Yes; 1-indexed pages and line numbers propagate through to student citation modals.

8. Is deletion safe? No, because document deletion is not yet exposed via the API.

9. Is it secure for official university use? No; authentication and access control are missing on the upload endpoint.

10. Top 5 Improvements: (1) Upload RBAC Auth, (2) Document Deletion API, (3) Async Ingestion Queue, (4) Persistent Hash Registry, (5) OCR & PDF Table Extraction.